

Scaling Code Understanding for Introductory Programming in the AI Era

Ashley Ge Zhang
University of Michigan
Ann Arbor, Michigan, USA
gezh@umich.edu

Abstract

Scaling programming education has become a significant challenge due to rapidly increasing learning needs and limited instructional resources. With the numerous data generated in programming education, scaling code understanding is crucial for instructors to make informed decisions, such as providing feedback and adapting course materials. However, this is impractical due to the wide variation among students' code and the complexity of activities like tracking coding progress and analyzing solution trajectories. Artificial Intelligence (AI) has great potential for analyzing large volumes of data, but it also presents difficulties in making this data quickly understandable for humans. My research aims to scale code understanding in meaningful ways for introductory programming through combining techniques from AI and visualization. I design and develop interactive systems that enable instructors navigate large, complex collections of code and identify patterns, thereby enhancing teaching activities and improving learning outcomes.

CCS Concepts

• **Human-centered computing** → **Interactive systems and tools**; • **Applied computing** → **Education**; • **Software and its engineering**;

Keywords

Programming Education, Code Visualization

ACM Reference Format:

Ashley Ge Zhang. 2025. Scaling Code Understanding for Introductory Programming in the AI Era. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '25)*, April 26–May 01, 2025, Yokohama, Japan. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3706599.3721095>

I am a fourth-year PhD student at the University of Michigan and expect to propose thesis in one year. I have no experience attending doctoral consortium. My work aims to invent new techniques to scale coding understanding for Computer Science Education. These systems scale programming instructors' effort in large introductory courses by reducing high dimensional-data like code to a low-dimensional space, thus support instructors to give students personalized feedback in large classes.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '25, Yokohama, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1395-8/25/04

<https://doi.org/10.1145/3706599.3721095>

1 Introduction

Scaling computer science education has become a major challenge, especially with the growing demand for programming skills and limited instructional resources [3, 4, 12, 14]. Introductory programming courses could enroll hundreds or thousands of students, while only a few instructors and teaching assistants are available to support them [22]. Despite the benefits of timely, personalized, formative feedback for students, it is unlikely that instructors have the capacity to provide individualized feedback at scale [2, 6, 10, 17–19].

To support the scaling of computer science education, my research focuses on one critical teaching activity — programming exercises. These exercises, where students have a problem description and write solutions from starter code, provide hands-on practice, assess understanding, and help identify concepts where students struggle [8, 16]. For instructors to provide high-quality feedback, they need a deep understanding of students' code and related activities, such as coding progress and solution trajectories. However, fully understanding students' code submissions, both in real-time and asynchronous settings, presents significant challenges [9, 24].

A single programming exercise can generate thousands of varied submissions [9], making it time-consuming for instructors to identify common approaches and mistakes by reading each submission. Prior work has explored automated grading and clustering methods to reduce the amount of code instructors must review [1, 9]. However, these methods are often limited to code that is free of syntax errors and can be executed, making them unsuitable for incomplete or incorrect code samples, which are common when monitoring students' progress. If taking the temporal aspect into consideration, understanding students' incremental changes and the direction of their solving approach is crucial [21]. Instructors also need efficient ways to provide feedback and understand how students respond to it, both during in-class exercises and after-class assignments [5, 9]. Existing approaches often fall short in supporting these needs.

My research aims to design systems that scale code understanding for instructors through AI assistance. Recent advancements in AI, such as large language models (LLMs), have shown promise in handling complex tasks quickly, making them suitable for helping instructors analyze code, identify mistakes, generate feedback, and interpret students' coding trajectories. However, relying solely on AI can overwhelm instructors with information and reduce their control over teaching. Thus, there is a need to combine AI and human effort effectively, enabling fast, customizable, and intuitive code understanding.

My work focuses on designing systems that scale code understanding through a human-AI collaborative lens. It addresses three key aspects: (1) monitoring students' coding progress, (2) identifying patterns in code, and (3) understanding variation among code

submissions. These systems provide scalable visual representations with AI support, reducing the number of samples instructors need to review, combining common elements while highlighting variations, and dynamically visualizing students' incremental progress. In the following sections, I introduce systems developed during my doctoral work and discuss future directions in this area.

2 Research Contribution

2.1 VizProg: Visualizing Students' Coding Progress On A 2D Map View

In in-class coding exercises, monitoring students' coding progress helps instructors identify common misconceptions and spot students who are struggling [11]. Understanding students' incremental changes provides instructors with insights into their problem-solving approaches. However, monitoring students' coding progress in real-time is challenging because of large course size and the limited time instructors have [24]. To facilitate understanding students' coding progress and problems in in-class programming exercises, I introduced VizProg (Figure 1), a system that visualizes the coding trajectory of students — depicting the evolution of their solutions from scratch — in large-scale programming courses [24]. VizProg is inspired by the idea of viewing a real world map, where people can clearly see the destinations, the start point, and the possible paths in between. VizProg employs a 2D map with dynamic dots, each representing an individual student's coding progress, thereby enabling instructors to rapidly identify common patterns in problem-solving approaches. For instance, students may adopt diverse strategies and make distinct errors when solving an exercise, with some nearing the correct solution and requiring minimal assistance, while others may be further from solving it and in need of more extensive support [20]. In VizProg, when students are working on their code, instructors see the dots, which represents students' coding progress, moving from left to the right, showing that students are writing from an empty solution to a complete solution. On the right side of the map in VizProg, different correct solutions are displayed at different positions of the y-axis. Instructors can see students moving toward different directions, showing that they approach the problem in different ways.

A comparison study showed that VizProg can help participants 1) discover more than twice as many student misunderstandings, and 2) find the misunderstandings with less than half of the time and fewer interactions. Additionally, participants were able to identify patterns from students' coding progress which were not found with existing tools. For instance, some students were wrangling at the middle of the map and never reached the finish line in the map, which means they have no idea how to solve the problem and need guidance. Some students were working on an approach and their trajectories suddenly jumped to a totally different approach, which could be a sign of copy-paste from other students' solutions.

2.2 RunEx: Augmenting Regular-Expression Code Search with Runtime Values

While VizProg introduces a novel way to monitor coding progress through visual representations of students' coding status, it's also important for instructors to have control over the information

displayed and the ability to customize results. Instructors may need to find specific patterns or anti-patterns in students' code, such as tracking the correct use of concepts, identifying common mistakes, or spotting anti-patterns that pass test cases but go against the principles taught in class. These needs require expressive queries that go beyond text-based search. Given that code samples from introductory courses are often small and self-contained, this allows for searching across runtime values by executing the code.

We introduced RunEx, a novel code search tool that enables programming instructors to quickly search for class-wide pattern in large codebases of students' code. In RunEx, we proposed a code search syntax that augment regular expressions with run time values. Users can add specifications for matching runtime values in addition to the original regular expression, so that they get greater expressiveness and precision without losing any of the flexibility of regular expressions for code search. The new syntax uses '«' and '»' to denote the start and the end region that specify runtime constrains. The content inside of the region '«»' can be one of the three types: 1) a value, 2) a **lambda** expression, or 3) empty (which will match any runtime value). If it is a value, RunEx will match any expression inside the region that evaluate to that value. For instance, **range**(«<40>») matches all of the following:

- **range**(40)
- **range**(10+30)
- **x = 40**
 range(x)

To create more expressive expressions, users can use a **lambda** expression that accepts two arguments — a runtime value and a string with the expression's contents — and return a boolean. For example, **x=x+«lambda v: type(v) is list»>>** is equivalent to expressions of **x=x+V** where V is list variable. More details can be found in the original paper [23].

To assist instructors to quickly create the search queries with little prior knowledge, we designed a user interface for RunEx. In the RunEx's user interface (Figure 2), instructors can create search queries by simply brushing and selecting code lines, and typing in the desired search criteria. Instructors can apply four types of set operations — intersection, union, difference, and complement — on the search queries to filter and group students' code solutions.

A comparison study showed that RunEx significantly enhances participants' abilities in three ways: 1) identifying mistakes and patterns with greater accuracy, 2) easily creating search queries with minimal prior knowledge, and 3) fully expressing their intentions in search queries.

2.3 CFlow: Supporting Semantic Flow Analysis of Students' Code in Programming Problems at Scale

CFlow scales code understanding by visualizing variation among large-scale student code submissions [25]. Variations can occur at different levels: some code samples vary in approach, resulting in entirely different solutions, while others differ only in specific lines, sharing common parts. Existing work on clustering code [9, 13] often overlooks these levels of variation. They either cluster based on entire code samples, missing subtle differences and creating too

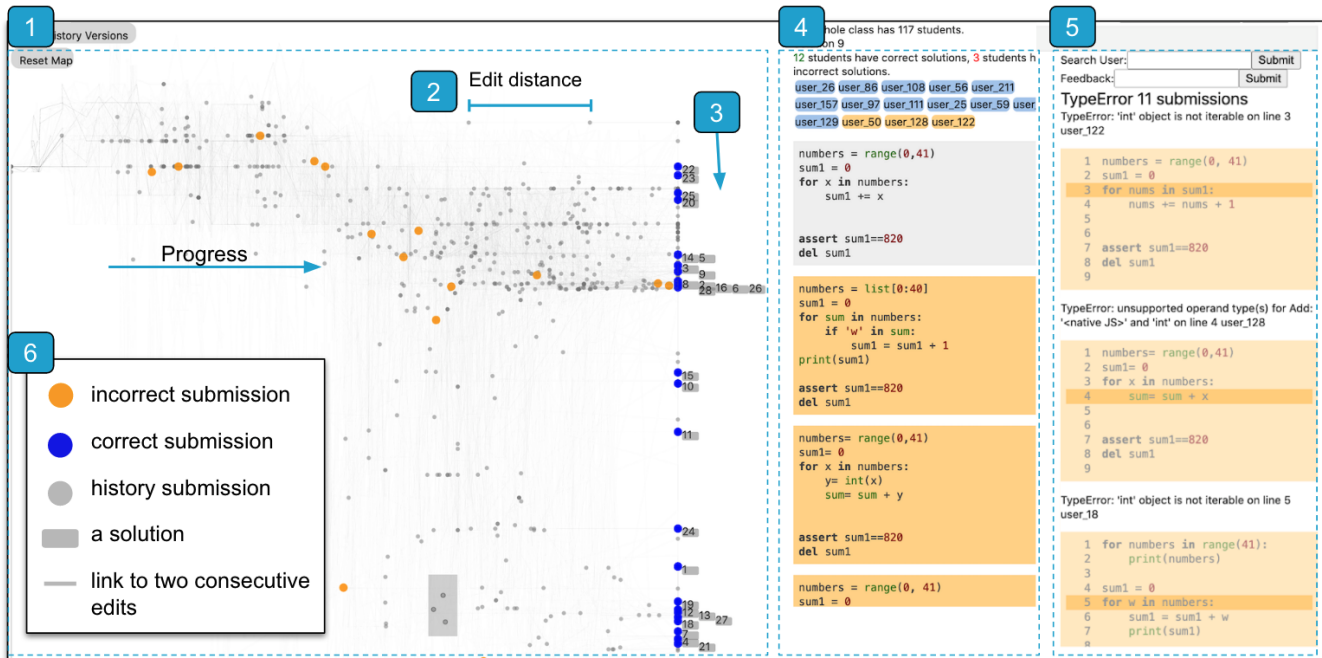


Figure 1: The user interface of VizProg. This figure is adapted from the original paper [24].

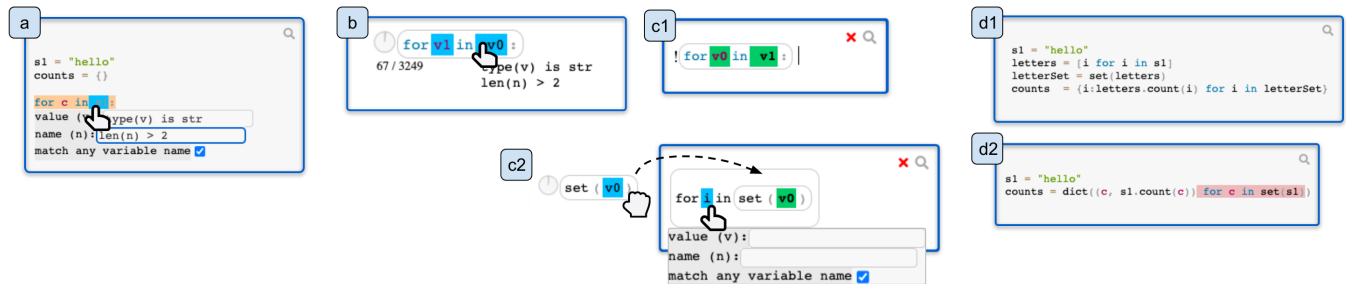


Figure 2: The user interface of RunEx. This figure is adapted from the original paper [23].

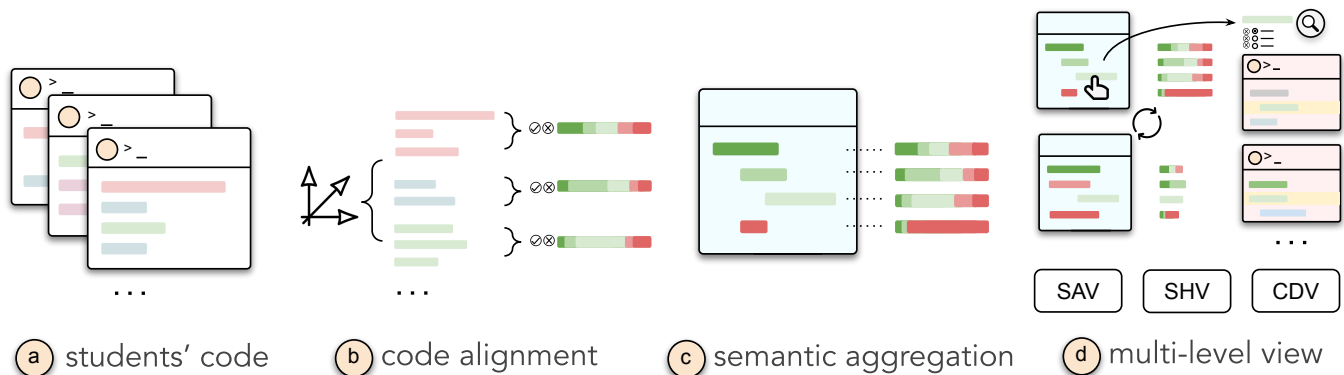


Figure 3: CFLOW's algorithm. This figure is adapted from the original paper [25]

many clusters, or focus on individual code statements, losing the context of the overall solution.

To bridge this gap, we introduce CFlow (Figure 3), a system that visualizes large-scale students' solutions of a programming exercise in an aggregated code framework, that presents the intermediate-level information about code. In CFlow's algorithm, students' code solutions are divided into code lines. All the code lines are clustered using CodeBert [7]. Code lines for similar purposes are clustered in one group. After that, we aligned the clusters back to a code framework following the order from most code solutions. CFlow's user interface has three main panels (Figure 3.d): 1) a semantic aggregated view, 2) a histogram view, and 3) a code detailed view. In the semantic aggregated view, each code line element represents a set of code lines that have the same purpose in the code. The code line elements are ordered as in the same way as they appear in students' solutions. Within each code line element, the code lines represent various implementations. In the histogram view, each code line element has a histogram alongside with it, with the solution's correctness color coded. Instructors can navigate and filter the solution set by clicking either the histogram or the code line elements. CFlow allows instructors to easily identify incorrect combinations of code statements — while individual statements may be correct, students must combine them properly to achieve a correct solution.

In the evaluation study, I compared CFlow to a combination of OverCode and RunEx. Our results show that CFlow help participants identify misconceptions with less time and greater accuracy. Additionally, CFlow fosters an exploratory approach to view students' code instead of having expectation of what approaches and mistakes students might have.

3 Future Work

My previous projects have primarily focused on helping instructors understand students' code, without exploring how scaling code understanding in introductory programming affects interactions between instructors and students. An important future question is how a coding progress monitoring dashboard can enhance the feedback loop between instructors and students.

Feedback is a critical factor in students' learning outcomes [18], and instructors should provide high-quality feedback in real-time during in-class exercises. However, this is challenging due to the complexity and constant changes in students' code, as well as the limited attention instructors can give. Recent advances in LLM agents show promise for facilitating and even automating this process [15]. A virtual teaching assistant (TA) powered by LLMs can quickly generate feedback at scale. However, there are drawbacks: instructors may struggle to ensure that interactions between students and LLM TAs align with course goals, and as class sizes increase, it becomes more challenging for instructors to monitor the performance of each LLM TA.

I propose a system, sketched in Figure 4, to support a human-AI collaborative feedback loop during in-class programming exercises. The system includes three main panels: a feedback panel, a progress panel, and a chat overview panel. It enables instructors to monitor students' coding progress in real-time, provide targeted feedback in collaboration with LLM TAs, and guide interactions between

students and LLM TAs. In the feedback panel, instructors can define the content and style of feedback across three dimensions: concepts to cover, message types, and feedback timing. Instructors can set these parameters before class for different student needs. For example, for students who are struggling, LLM TAs might first check their understanding of key concepts and provide short, immediate prompts to encourage critical thinking without adding stress. Each student is paired with an LLM TA, receiving feedback either upon request or when the system detects a lack of progress, such as no meaningful progress after one minute. The progress panel displays each student's coding progress as a timeline, showing progress toward a correct solution from left to right. Color-coded avatars indicate active participation, allowing instructors to monitor progress in real-time. As interactions between students and LLM TAs generate extensive chat logs, the chat overview panel offers a high-level view of these conversations. It helps instructors answer three key questions: (1) Does the conversation align with course goals? (2) How do students respond to feedback? (3) Does the feedback help students progress?

The second direction I plan to explore is to add semantic meaning to the map view presented in VizProg [24]. With the new map view, instructor can answer the following questions. **What are students' mistakes at this point?** Students' mistakes change constantly while they write code. Understanding the mistakes they currently have helps determine if students are struggling and whether they need additional support. **How many students are stuck on a mistake and do not know how to solve it?** If a student gets stuck on a particular issue, the instructor can provide targeted guidance to help them progress. Conversely, if most students can resolve the mistake independently, the instructor can focus on other areas. Knowing the number of students struggling with the same issue helps prioritize instructional interventions. **What programming concept is relevant to the mistakes?** Identifying the programming concepts related to mistakes enables instructors to focus their explanations and teaching on those concepts. This targeted approach can help students understand and overcome specific challenges more effectively. **How far are students from solving the problem successfully?** This question has two parts. The first aspect is their **overall progress**. Determine how close students are to completing the programming exercise. Students with the same mistake may be at different stages of their progress. A system should provide a visual representation of progress, such as a map view, that can also be adjusted to highlight mistakes. The second aspect is their **Mistake Resolution**. Assess how close a student is to correcting a specific mistake. Instead of manually checking the exact code content, the system could use keystroke events and other indicators to predict if a student is likely to solve the mistake on their own.

While this proposed system improves comprehension of students' code for a single exercise, I aim to expand it to multiple exercises in future work. It is acceptable for students to make mistakes as long as they do not repeat them. Since different exercises can share the same concepts, aggregating performance across multiple exercises will give instructors a more comprehensive understanding of students' struggles.

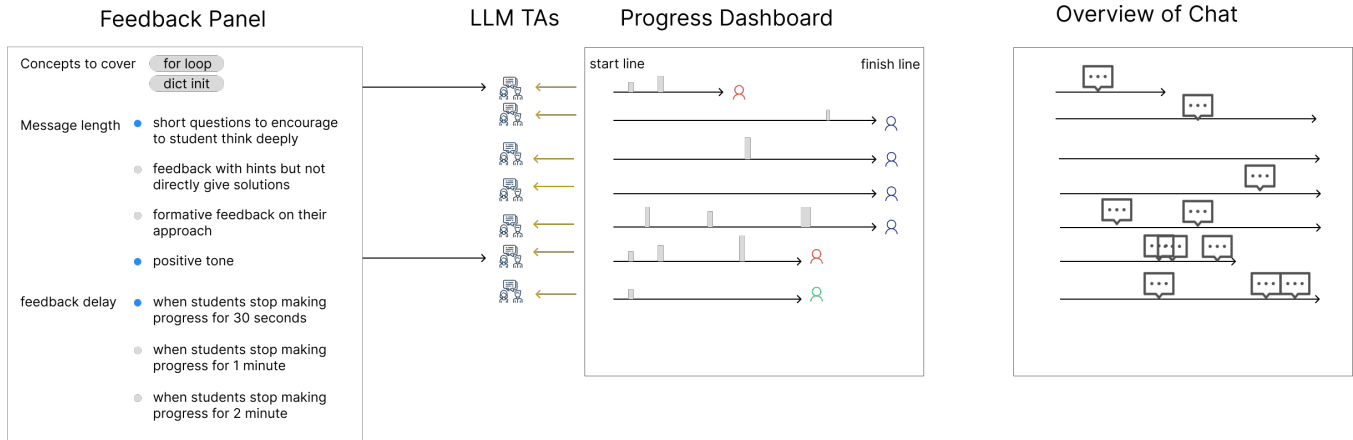


Figure 4: The proposed system's user interface.

Acknowledgments

References

- [1] Douglas S Blank, David Bourgin, Alexander Brown, Matthias Bussonnier, Jonathan Frederic, Brian Granger, Thomas L Griffiths, Jessica Hamrick, Kyle Kelley, M Pacer, et al. 2019. nbgrader: A tool for creating and grading assignments in the Jupyter Notebook. *The Journal of Open Source Education* 2, 11 (2019).
- [2] Miguel A Brito and Filipe De Sá-Soares. 2014. Assessment frequency in introductory computer programming disciplines. *Computers in Human Behavior* 30 (2014), 623–628.
- [3] Tracy Camp, W Richards Adrion, Betsy Bizot, Susan Davidson, Mary Hall, Susanne Hambrusch, Ellen Walker, and Stuart Zweben. 2017. Generation CS: the growth of computer science. *ACM Inroads* 8, 2 (2017), 44–50.
- [4] Parmit K Chilana, Rishabh Singh, and Philip J Guo. 2016. Understanding conversational programmers: A perspective from the software industry. In *Proceedings of the 2016 CHI conference on human factors in computing systems*. 1462–1472.
- [5] Loris D'antoni, Dileep Kini, Rajeev Alur, Sumit Gulwani, Mahesh Viswanathan, and Björn Hartmann. 2015. How can automatic feedback help students construct automata? *ACM Transactions on Computer-Human Interaction (TOCHI)* 22, 2 (2015), 1–24.
- [6] Nejoed Eltegeani and Laurie Butgereit. 2015. Attributes of students engagement in fundamental programming learning. In *2015 International Conference on Computing, Control, Networking, Electronics and Embedded Systems Engineering (ICCNTEE)*. IEEE, 101–106.
- [7] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).
- [8] Alex Gerdes, Johan T Jeuring, and Bastiaan J Heeren. 2010. Using strategies for assessment of programming exercises. In *Proceedings of the 41st ACM technical symposium on Computer science education*. 441–445.
- [9] Elena L Glassman, Jeremy Scott, Rishabh Singh, Philip J Guo, and Robert C Miller. 2015. OverCode: Visualizing variation in student solutions to programming problems at scale. *ACM Transactions on Computer-Human Interaction (TOCHI)* 22, 2 (2015), 1–35.
- [10] Anabela Gomes and Antonio Mendes. 2014. A teacher's view about introductory programming teaching and learning: Difficulties, strategies and motivations. In *2014 IEEE Frontiers in Education Conference (FIE) Proceedings*. IEEE, 1–8.
- [11] Philip J Guo. 2015. Codeopticon: Real-time, one-to-many human tutoring for computer programming. In *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology*. 599–608.
- [12] Philip J Guo. 2017. Older adults learning computer programming: Motivations, frustrations, and design opportunities. In *Proceedings of the 2017 chi conference on human factors in computing systems*. 7070–7083.
- [13] Andrew Head, Elena Glassman, Gustavo Soares, Ryo Suzuki, Lucas Figueredo, Loris D'Antoni, and Björn Hartmann. 2017. Writing reusable code feedback at scale with mixed-initiative program synthesis. In *Proceedings of the Fourth (2017) ACM Conference on Learning@ Scale*. 89–98.
- [14] US Department of Labor, Bureau of Statistics Staff, and United States. Bureau of Labor Statistics. 2000. *Occupational outlook handbook*. Bernan Press (PA).
- [15] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*. 1–22.
- [16] Arnold Pears, Stephen Seidman, Lauri Malmi, Linda Mannila, Elizabeth Adams, Jens Bennedsen, Marie Devlin, and James Paterson. 2007. A survey of literature on the teaching of introductory programming. *Working group reports on ITiCSE on Innovation and technology in computer science education* (2007), 204–223.
- [17] Leo Porter and Daniel Zingaro. 2014. Importance of early performance in CS1: two conflicting assessment stories. In *Proceedings of the 45th ACM technical symposium on Computer science education*. 295–300.
- [18] Alexander Repenning, Ashok Basawapatna, and Nora Escherle. 2016. Computational thinking tools. In *2016 IEEE symposium on visual languages and human-centric computing (VL/HCC)*. IEEE, 218–222.
- [19] Álvaro Santos, Anabela Gomes, and António Mendes. 2013. A taxonomy of exercises to support individual learning paths in initial programming learning. In *2013 IEEE Frontiers in Education Conference (FIE)*. IEEE, 87–93.
- [20] Valerie J Shute. 2008. Focus on formative feedback. *Review of educational research* 78, 1 (2008), 153–189.
- [21] Lisa Wang, Angela Sy, Larry Liu, and Chris Piech. 2017. Deep knowledge tracing on programming exercises. In *Proceedings of the fourth (2017) ACM conference on learning@ scale*. 201–204.
- [22] Christopher Watson and Frederick WB Li. 2014. Failure rates in introductory programming revisited. In *Proceedings of the 2014 conference on Innovation & technology in computer science education*. 39–44.
- [23] Ashley Ge Zhang, Yan Chen, and Steve Oney. 2023. RunEx: Augmenting Regular-Expression Code Search with Runtime Values. In *2023 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, 139–147.
- [24] Ashley Ge Zhang, Yan Chen, and Steve Oney. 2023. VizProg: Identifying Misunderstandings By Visualizing Students' Coding Progress. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–16.
- [25] Ashley Ge Zhang, Xiaohang Tang, Steve Oney, and Yan Chen. 2024. CFlow: Supporting Semantic Flow Analysis of Students' Code in Programming Problems at Scale. *arXiv preprint arXiv:2404.10089* (2024).

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009